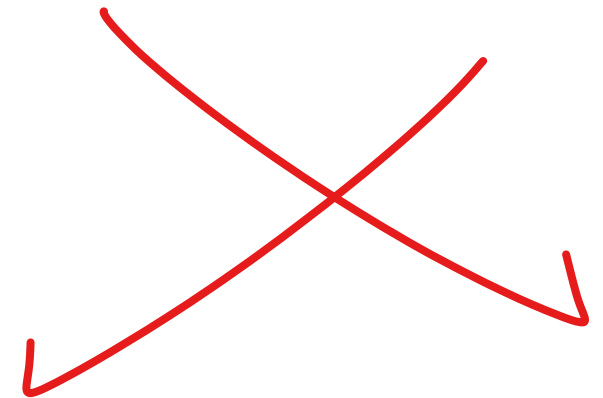


Application of Stack



INFIX POSTFIX PRECEDENCE ASSOCIATIVITY



Understanding infix, prefix and postfix

- **Infix expression:**

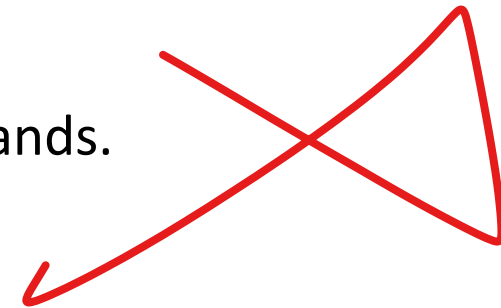
- The expression of the form **a op b**.
- An operator is **in-between every pair** of operands.
- Example: $5 + 8$

- **Postfix expression:**

- The expression of the form **a b op**.
- An operator follows every pair of operands.
- Example: $5\ 8\ +$

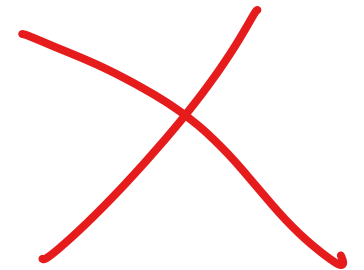
- **Prefix expression:**

- The expression of the form **op a b**.
- When an operator is present before every pair of operands.
- Example: $+ 5\ 8$



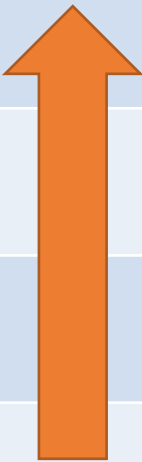
Mathematical Calculations: Precedence

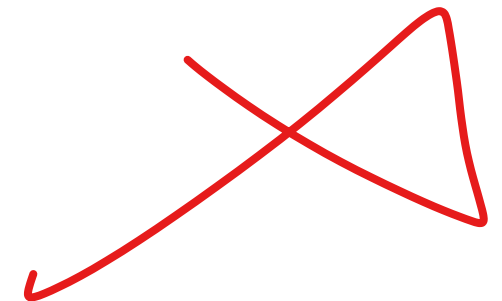
- The way we are use to writing expressions is known as infix notation
 - What does $3 + 2 * 6$ equal? Ans: 15
 - What does $3 * 2 + 6$ equal? Ans: 12
- The **precedence** of operators affects the order of operations.
- A mathematical expression cannot simply be evaluated left to right.



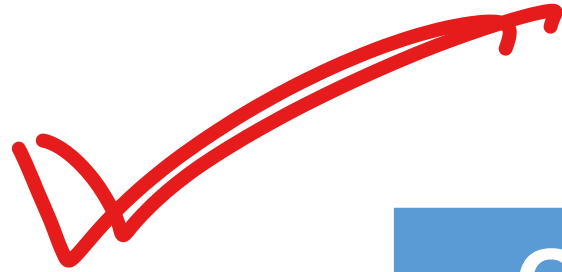
Mathematical Calculations: Precedence

RECALL BODMAS

Operator	Precedence
$()$	 HIGHEST
$\wedge$	
$/ *$	
$+ -$	
	LOWEST



Mathematical Calculations: Associativity



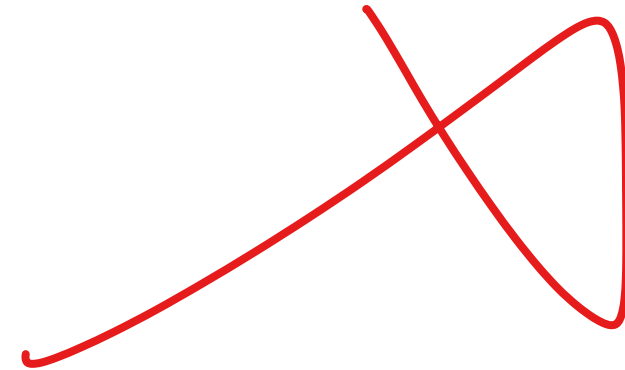
Operator	Associativity
$()$	DO NOT HAVE IT
$\wedge$	R to L
$/ *$	L to R
$+ -$	L to R

Understanding Associativity

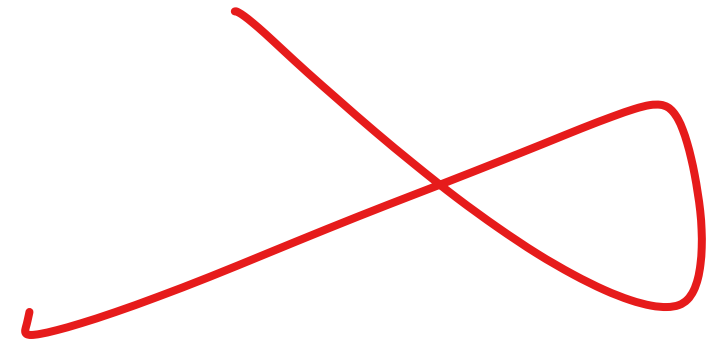
- **Associativity:** It describes how operations of the **same precedence** are grouped when there are no parentheses to dictate the order.
- Two types of associativity:
 - **Right-to-left (or right-associative):** This means you perform the operations from right to left.
 - Let us consider 2^3^2 ✓
 - **Left-to-right (or left-associative):** This means you perform the operations from left to right.
 - Let us consider $5-3-2$

Summary

- We have understood the following concepts:
 - Prefix, Infix, Postfix
 - Precedence
 - Associativity
- Now let us put these concepts to use.

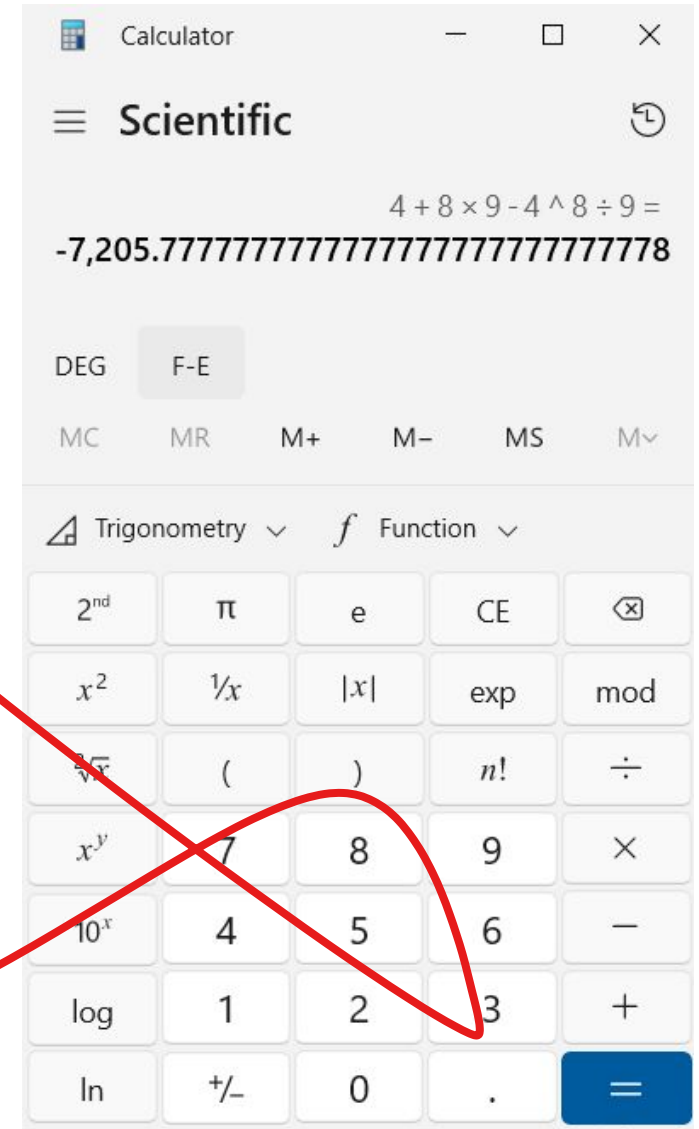


LET'S MAKE A CALCULATOR



Calculator

- We know how any device can take in strings and store it in memory
 - Recall from DLD, SPL
- We can give a string with mathematical characters
 - “4+3-3^12+(2-3)”
- What can you do to process this string and get the result?



Infix to Postfix Conversion

Brackets are used in ~~infix~~ notation

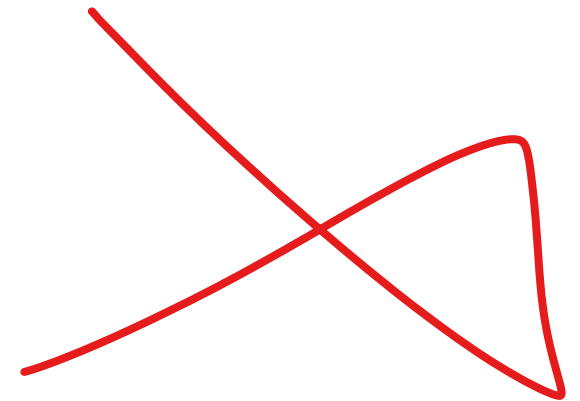
- Note that unlike infix, postfix notations **do not have brackets**
- There is **no requirement** for brackets

Doing it on a piece of paper

- No need to use a stack during conversion, but need to use stack during evaluation

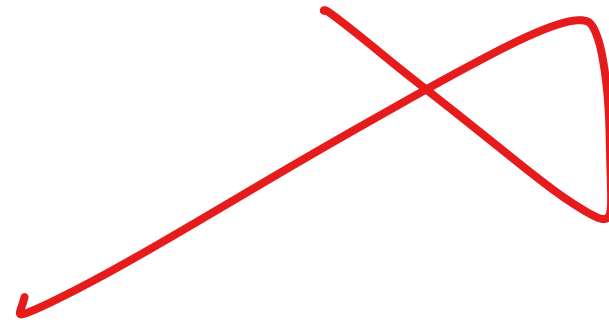
Using a stack

- For the conversion algorithm we need to use a stack
- It will be a character stack
- For evaluation we will also need to use a stack



Infix to Postfix Conversion

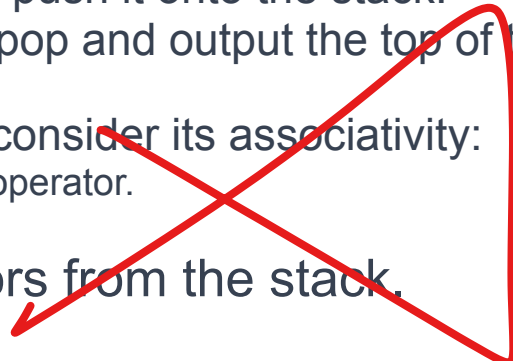
Let us develop a pseudocode where a stack will be used.



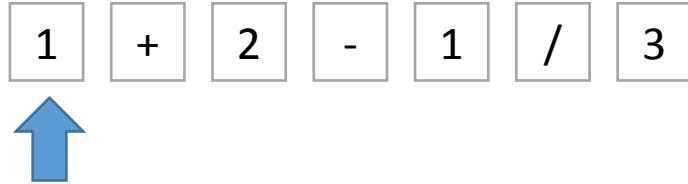
Infix to Postfix Conversion

1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

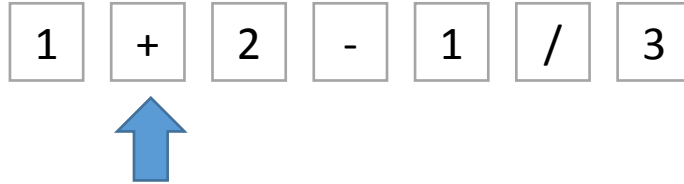
- 
1. Scan the infix expression from left to right.
 2. If the scanned character is an operand, output it.
 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
 4. At the end of the expression, pop and output all remaining operators from the stack.
- 

Infix to Postfix Conversion



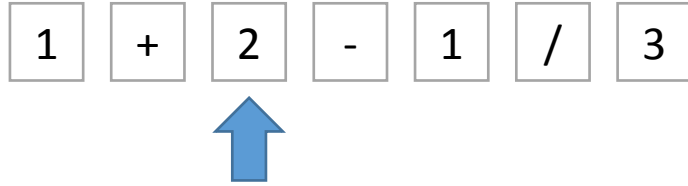
- ➔
1. Scan the infix expression from left to right.
 2. If the scanned character is an operand, output it.
 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
 4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



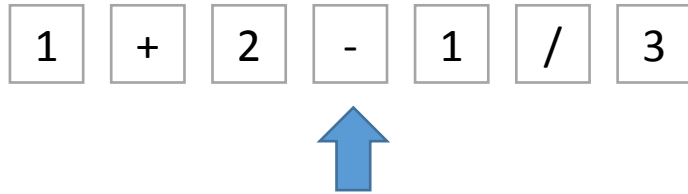
- ➔
1. Scan the infix expression from left to right.
 2. If the scanned character is an operand, output it.
 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
 4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



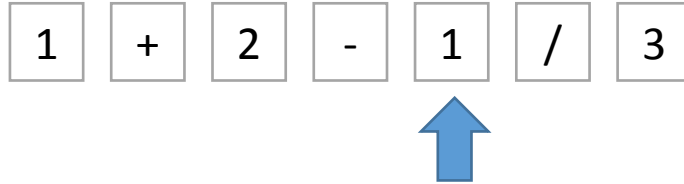
- ➔
1. Scan the infix expression from left to right.
 2. If the scanned character is an operand, output it.
 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
 4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



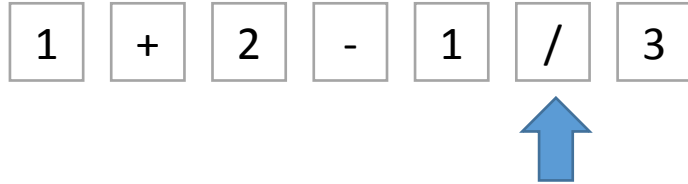
- ➔
1. Scan the infix expression from left to right.
 2. If the scanned character is an operand, output it.
 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
 4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



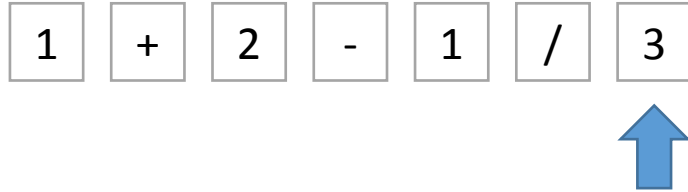
- ➔
1. Scan the infix expression from left to right.
 2. If the scanned character is an operand, output it.
 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
 4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



- ➔
1. Scan the infix expression from left to right.
 2. If the scanned character is an operand, output it.
 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
 4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



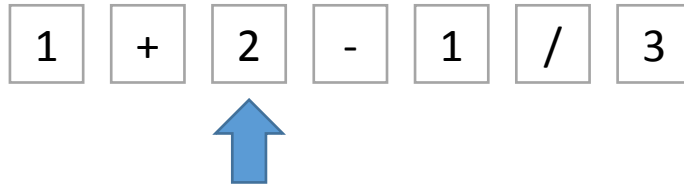
- ➔
1. Scan the infix expression from left to right.
 2. If the scanned character is an operand, output it.
 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
 4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

1 + 2 - 1 / 3

1. Scan the infix expression from left to right.
- ➔ 2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



Console(or an array):

(previous characters)2

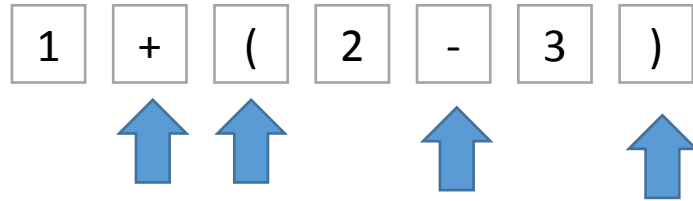
1. Scan the infix expression from left to right.
- ➔ 2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

1 + (2 - 3)

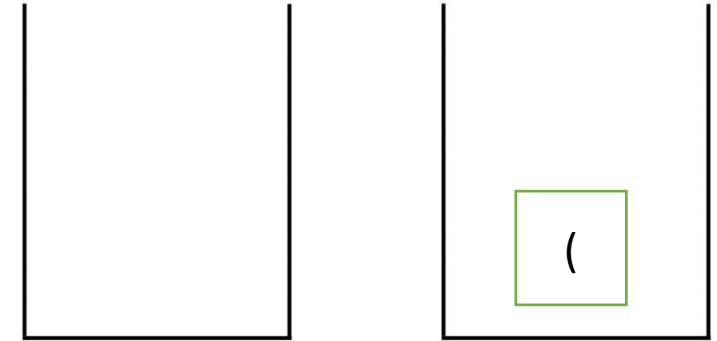
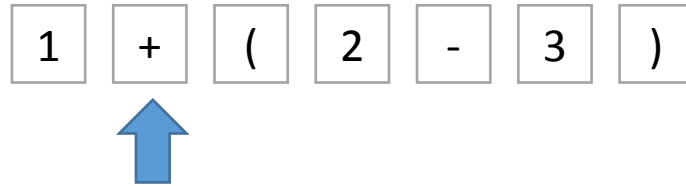
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
- ➔ 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



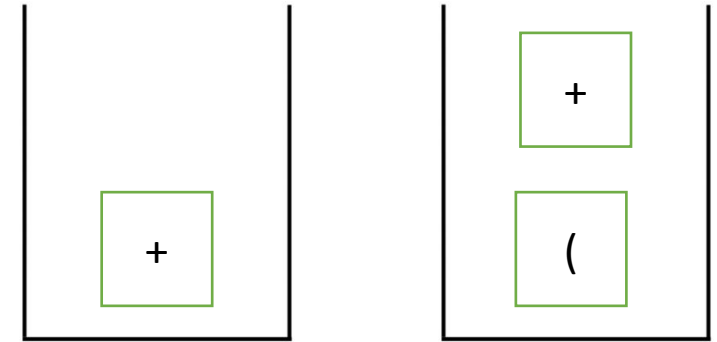
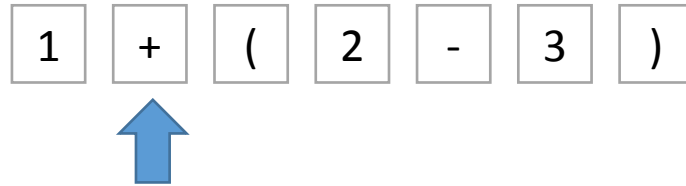
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
- ➔ 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming operator onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



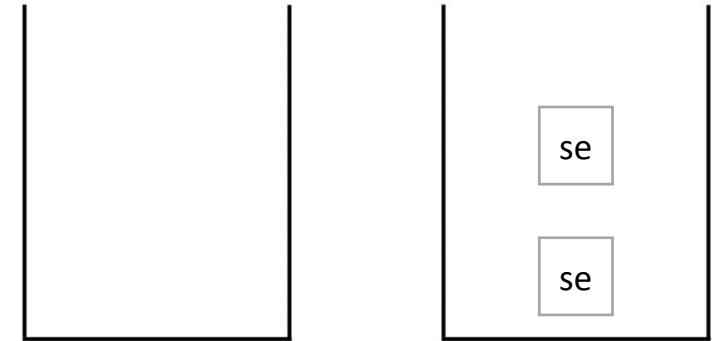
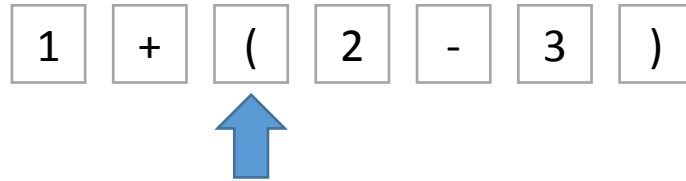
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



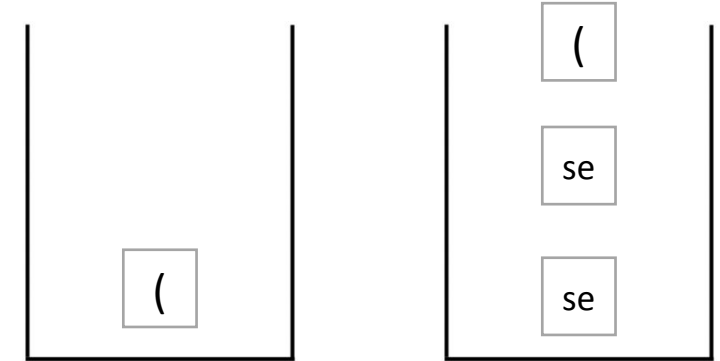
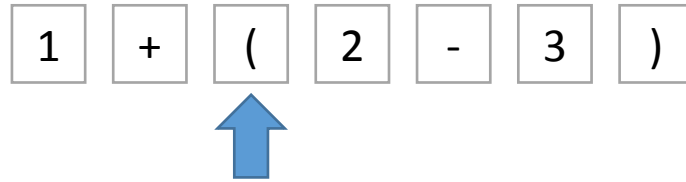
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming symbol is '(', push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming symbol is ')', pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

1 + (2 - 3)



Console(or an array):

se se

se
se
(

1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')'**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

1 + (2 - 3)



Console(or an array):

se se se

se
(

1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')'**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

1 + (2 - 3)



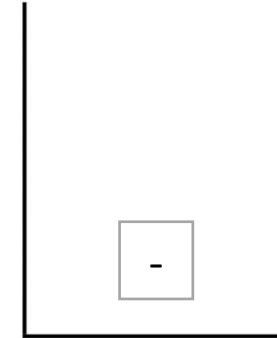
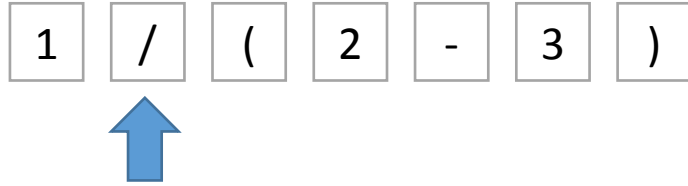
Console(or an array):

se se se se

(

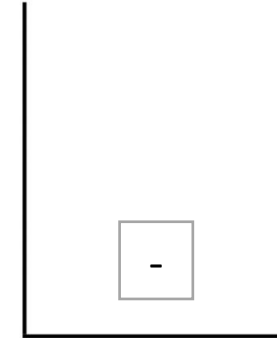
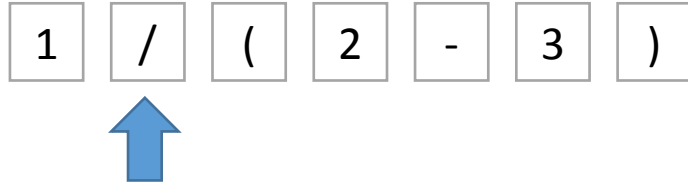
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator has higher precedence than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



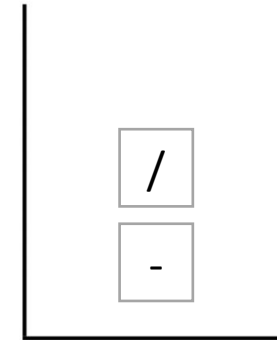
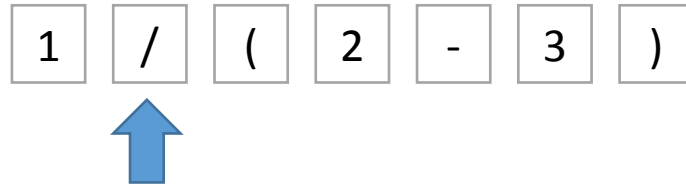
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



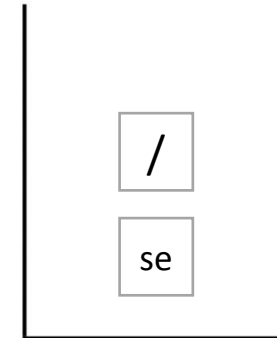
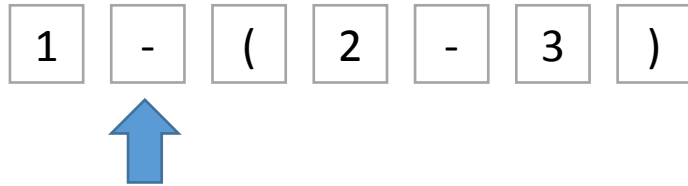
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



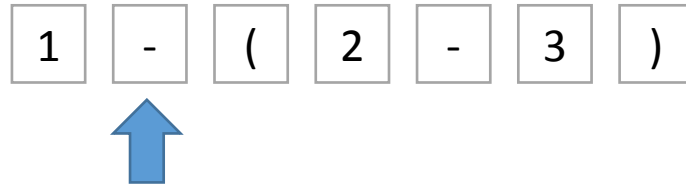
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator has lower precedence than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



Console(or an array):

se se

/

se

1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

1 - (2 - 3)



Console(or an array):

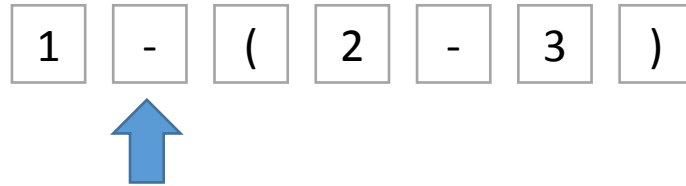
se se /

se

1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')'**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

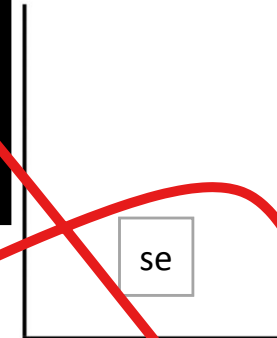
1 - (2 - 3)



Console(or an array):

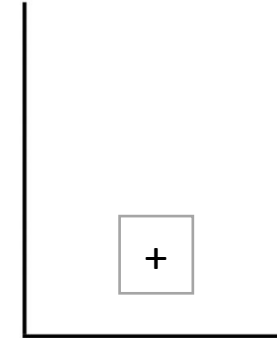
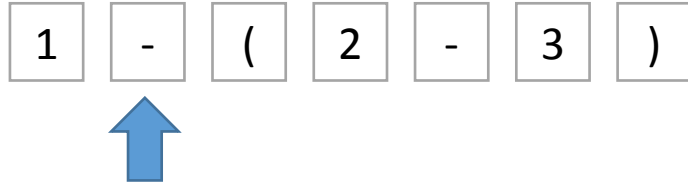
se se /

se



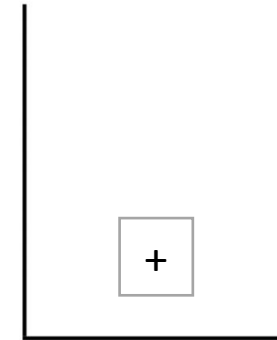
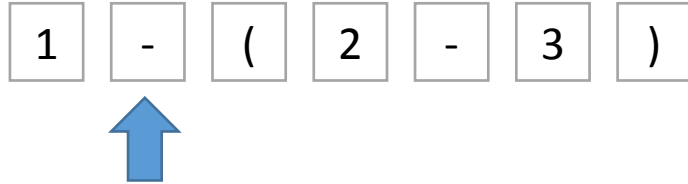
1. Scan the infix expression from left to right.
 2. If the scanned character is an operand, output it.
 3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
 4. At the end of the expression, pop and output all remaining operators from the stack.
- 

Infix to Postfix Conversion



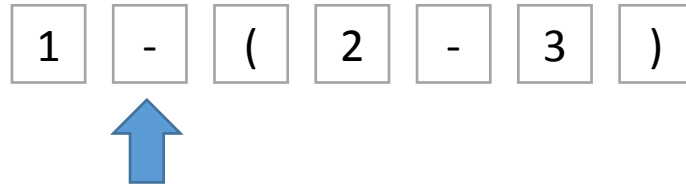
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, pop and output the top of the stack, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, **pop and output the top of the stack**, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



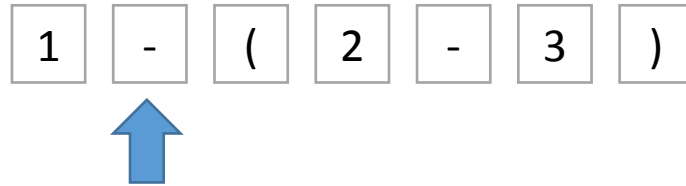
Console(or an array):

se se



1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, **pop and output the top of the stack**, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

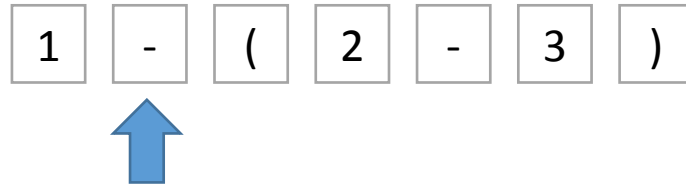


Console(or an array):

se se +

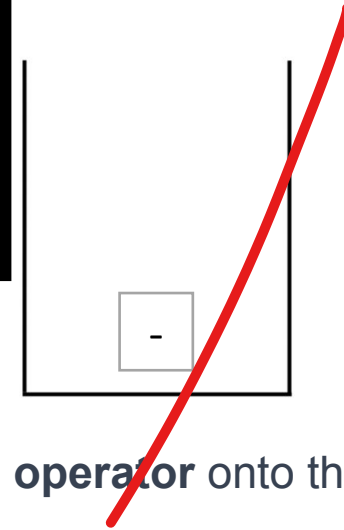
1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, **pop and output the top of the stack**, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion



Console(or an array):

se se +



1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, **pop and output the top of the stack**, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining operators from the stack.

Infix to Postfix Conversion

1 - (2 - 3)



1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, **pop and output the top of the stack**, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
- ➡ 4. At the end of the expression, pop and output all remaining **operators** from the stack.

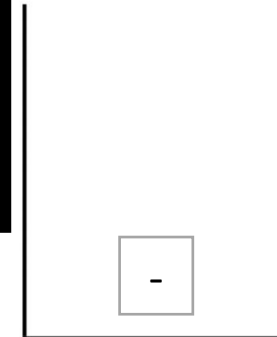
Infix to Postfix Conversion

1 - (2 - 3)



Console(or an array):

se se se



1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, **pop and output the top of the stack**, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining **operators** from the stack.

Infix to Postfix Conversion

1 - (2 - 3)



Console(or an array):

se se se -

1. Scan the infix expression from left to right.
2. If the scanned character is an operand, output it.
3. If the scanned character is an operator or a parenthesis:
 1. If the stack is empty or contains a left parenthesis on top, push the incoming **operator** onto the stack.
 2. If the incoming **symbol is '('**, push it onto the stack.
 3. If the incoming **symbol is ')''**, pop the stack and output the operators until a '(' is found.
 4. If the incoming operator **has higher precedence** than the top of the stack, push it onto the stack.
 5. If the incoming operator **has lower precedence** than the top of the stack, pop and output the top of the stack, then test the incoming operator against the new top of the stack.
 6. If the incoming operator has equal precedence with the top of the stack, consider its associativity:
 1. If it's left-associative, **pop and output the top of the stack**, then push the incoming operator.
 2. If it's right-associative, push the incoming operator.
4. At the end of the expression, pop and output all remaining **operators** from the stack.

Let us practice with some examples

- a) $2^3 \cdot 4 + 5 \cdot 7$

- Ans: $2 \ 3 \ 4 \wedge \wedge 5 \ 7 \ * \ +$

Infix to Postfix

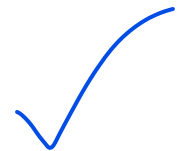
- b) $9 + 2 - 5 \cdot 6 / 3 + 2^3 / 4$

- Ans: $9 \ 2 \ + \ 5 \ 6 \ * \ 3 \ / \ - \ 2 \ 3 \ ^ \ 4 \ / \ +$



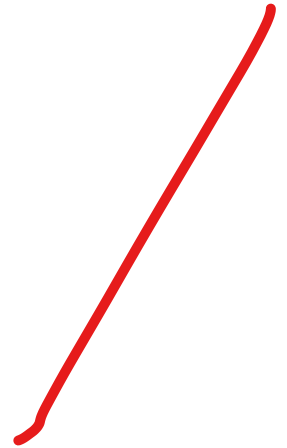
- c) $1 + 3^2 + (5 \cdot 6 - 4) \cdot 7$

- Ans: $1 \ 3 \ 2 \ ^ \ + \ 5 \ 6 \ * \ 4 \ - \ 7 \ * \ +$



We can convert from infix to postfix

- Now let us evaluate the postfix expression
- Another stack will be required



We can convert from infix to postfix

- Now let us evaluate the postfix expression
- Another stack will be required

for each character in postfix expression do

- if it is an operand then push it onto the stack
- else if it is an operator (say, op) then
 - pop the stack for the right hand operand (say, b)
 - pop the stack for the left hand operand (say, a)
 - apply the operator to the two operands (result = a op b)
 - push the result onto the stack
- when the expression has been exhausted the result is the top (and only element) of the stack

Let us practice with some examples

in to post

- a) $2^3 \cdot 4 + 5 \cdot 7$

- Ans: $2 \ 3 \ 4 \ ^ \cdot 5 \ 7 \ * \ +$

- b) $9 + 2 - 5 \cdot 6 / 3 + 2^3 / 4$

- Ans: $9 \ 2 \ + \ 5 \ 6 \ * \ 3 \ / \ - \ 2 \ 3 \ ^ \ 4 \ / \ +$

- c) $1 + 3^2 + (5 \cdot 6 - 4) \cdot 7$

- Ans: $1 \ 3 \ 2 \ ^ \ + \ 5 \ 6 \ * \ 4 \ - \ 7 \ * \ +$



How to deal with 2 or more digits?

- The algorithm can handle multi-digit numbers just fine.
- When scanning from left to right, if you encounter a digit, keep scanning to the right until you encounter a non-digit character.
- All the digits scanned together form a single operand, which could be a multi-digit number.

How to deal with 2 or more digits?

- For example, consider the infix expression $12 + 34$. While scanning from left to right:

WE CAN USE OPERAND OBJECTS
WHILE CODING

1. You encounter 1 which is a digit. Keep scanning.
2. The next character is 2 which is **also a digit**. Therefore 12 forms a single operand.
3. The next character is + which is an operator. So you output the operand 12 and process the operator +.
4. Continue scanning. 3 is a digit. Keep scanning.
5. 4 is also a digit. So 34 forms the second operand.
6. In postfix notation, this expression becomes 12 34 +



Thank You

